

Java Junior Developer Assessment

Medium Level • OOP, Exceptions, Collections, equals() & hashCode()
60 Multiple-Choice Questions • Time: 90 minutes

Name: _____

Date: _____

Score: _____ / 60

Duration: 90 minutes

Instructions: Circle or write the letter of the single best answer for each question. Each question is worth 1 point (60 points total). No partial credit.

Section 1: Object-Oriented Programming (OOP)

1. Which OOP principle allows a subclass to provide a specific implementation of a method already defined in its superclass?

- A. Encapsulation
- ☒ B. Overriding (Polymorphism)
- C. Overloading
- D. Abstraction

2. What is the main purpose of encapsulation in Java?

- A. To allow multiple inheritance
- ☒ B. To hide internal state and require all interaction through an object's methods
- C. To speed up compilation
- D. To allow static methods only

3. Which keyword is used to prevent a class from being subclassed?

- A. static
- ☒ B. final
- C. private
- D. abstract

4. Which of the following best describes method overloading?

- A. Same method name, same parameters, different return type
- ☒ B. Same method name, different parameter list, in the same class
- C. Same method signature in a subclass
- D. Different method names, same parameters

5. What does the 'this' keyword refer to inside an instance method?

- A. The superclass instance
- B. The current class definition
- ☒ C. The current object instance
- D. A static reference to the class

6. Which statement about abstract classes is TRUE?

- A. They can be instantiated directly
- ☒ B. They can contain both abstract and concrete methods
- C. They cannot have constructors
- D. They cannot have instance fields

7. What is the key difference between an interface and an abstract class in modern Java (8+)?

- A. Interfaces cannot have any method bodies at all
- ☒ B. A class can implement multiple interfaces but extend only one class
- C. Abstract classes support multiple inheritance of state
- D. Interfaces can have constructors

8. Which access modifier makes a member accessible only within its own class?

- A. protected
- B. public
- ☒ C. private
- D. default (package-private)

9. What is polymorphism in Java primarily used for?

- A. Reducing memory usage of primitive types
- ☒ B. Allowing one interface/reference type to be used for different underlying object types
- C. Making variables immutable
- D. Preventing inheritance

10. In the statement `Animal a = new Dog();`, which type determines which overridden method is called at runtime?

- A. The compile-time (reference) type, Animal
- ☒ B. The runtime (actual object) type, Dog
- C. Neither, it causes a compile error
- D. It depends on the variable name

11. What happens if a subclass does not override an abstract method inherited from its superclass?

- A. Nothing, it compiles fine
- ☒ B. The subclass must also be declared abstract, or it fails to compile
- C. Java provides a default empty implementation automatically
- D. It throws a runtime exception

12. Which of these is NOT one of the four core OOP pillars?

- A. Inheritance
- B. Polymorphism
- ☒ C. Compilation
- D. Encapsulation

13. What is 'composition' in OOP design?

- ☒ A. Building a class by including instances of other classes as fields ('has-a' relationship)
- B. Extending a class using inheritance
- C. Overloading constructors
- D. Casting an object to its superclass type

14. Which keyword allows a class to inherit from another class in Java?

- A. implements
- ☒ B. extends
- C. inherits
- D. super

15. What is the purpose of a constructor in Java?

- A. To destroy an object
- ☒ B. To initialize a newly created object
- C. To override a method
- D. To define static utility methods

16. Given `interface Shape { double area(); }`, which is TRUE for a class implementing Shape?

- A. It may leave `area()` unimplemented if declared abstract
- ☒ B. It must implement `area()` unless the class itself is abstract
- C. It cannot have additional methods
- D. It cannot be extended further

Section 2: Exceptions

17. What is the superclass of all exceptions and errors in Java?

- A. Exception
- B. RuntimeException
- ☒ C. Throwable
- D. Error

18. Which of the following is a checked exception?

- A. NullPointerException
- B. ArrayIndexOutOfBoundsException
- ☒ C. IOException
- D. ArithmeticException

19. What must a method do if it can throw a checked exception and does not handle it?

- A. Nothing special is required
- ☒ B. Declare it using the 'throws' clause
- C. Catch it silently
- D. Convert it to an unchecked exception automatically

20. Which block always executes whether or not an exception is thrown (barring JVM exit)?

- A. catch
- B. try
- ☒ C. finally
- D. throw

21. What is the correct syntax to manually throw an exception?

- ☒ A. throw new IllegalArgumentException("bad value");
- B. throws new IllegalArgumentException();
- C. raise IllegalArgumentException();
- D. throw IllegalArgumentException;

22. Which of these is an unchecked exception (subclass of RuntimeException)?

- A. SQLException
- B. FileNotFoundException
- ☒ C. NullPointerException
- D. InterruptedException

23. What is exception chaining used for?

- ☒ A. Preserving the original cause while wrapping it in a new exception type
- B. Running multiple catch blocks in parallel
- C. Improving loop performance
- D. Disabling stack traces

24. In a try-with-resources statement, what requirement must the resource class satisfy?

- A. It must implement Serializable
- ☒ B. It must implement AutoCloseable (or Closeable)
- C. It must be declared final
- D. It must throw a checked exception in its constructor

25. What happens if a finally block contains a return statement?

- A. It is ignored by the compiler
- ☒ B. It can override/suppress a return or exception from the try/catch block
- C. It causes a compile-time error always
- D. It only runs if no exception occurred

26. Which statement about multi-catch (`catch (IOException | SQLException e)`) is TRUE?

- ☒ A. The caught exception types must not be related by subclassing to each other
- ☐ B. It is invalid Java syntax
- ☐ C. It only works with unchecked exceptions
- ☐ D. It requires each type to have the exact same superclass method signatures

27. What is a custom exception typically created by?

- ☐ A. Overriding the JVM's Throwable class directly
- ☒ B. Extending Exception or RuntimeException
- ☐ C. Implementing the Error interface
- ☐ D. Annotating a class with @Exception

28. What does 'exception propagation' mean?

- ☐ A. An exception being logged twice
- ☒ B. An unhandled exception moving up the call stack to the caller
- ☐ C. Converting an exception to a warning
- ☐ D. Catching the same exception in multiple catch blocks

29. Which best practice should generally be avoided when catching exceptions?

- ☐ A. Catching specific exception types
- ☐ B. Logging the exception with context
- ☒ C. Catching generic Exception (or Throwable) and silently ignoring it
- ☐ D. Using try-with-resources for closeable resources

30. What is the output behavior if an exception is thrown inside a catch block?

- ☐ A. It is automatically suppressed
- ☒ B. It propagates up unless handled by an enclosing try/catch or a following finally rethrows differently
- ☐ C. The JVM ignores it
- ☐ D. The original exception is re-thrown instead

31. Which of the following correctly describes 'Error' (e.g., `OutOfMemoryError`) in Java?

- ☐ A. It should always be caught and recovered from in business logic
- ☒ B. It represents serious problems that applications typically should not try to catch
- ☐ C. It is a checked exception
- ☐ D. It extends RuntimeException

32. What does the following code print?

```
public class Test {
    public static void main(String[] args) {
        try {
            System.out.print("A");
            throw new RuntimeException();
        } catch (RuntimeException e) {
            System.out.print("B");
        } finally {
            System.out.print("C");
        }
    }
}
```

- ☐ A. A
- ☐ B. AB
- ☒ C. ABC
- ☐ D. AC

Section 3: Collections Framework

33. Which interface does NOT extend Collection?

- A. List
- B. Set
- C. Map**
- D. Queue

34. Which List implementation provides fast random access via index?

- A. LinkedList
- B. ArrayList**
- C. TreeSet
- D. HashSet

35. Which Set implementation maintains elements in their natural sorted order?

- A. HashSet
- B. LinkedHashSet**
- C. TreeSet
- D. ArraySet

36. What is the key characteristic of a HashSet regarding element order?

- A. It preserves insertion order
- B. It does not guarantee any specific iteration order**
- C. It sorts elements automatically
- D. It allows duplicate elements

37. Which Map implementation preserves insertion order of its entries?

- A. HashMap
- B. TreeMap
- C. LinkedHashMap**
- D. Hashtable

38. What does `List.of(1, 2, 3)` return?

- A. A mutable ArrayList
- B. An immutable list that throws UnsupportedOperationException on modification**
- C. A LinkedList
- D. A null reference

39. Which of the following is true about ArrayList vs LinkedList for frequent insertions/removals in the middle of a large list?

- A. ArrayList is generally faster because it uses contiguous memory
- B. LinkedList is generally faster because it avoids shifting elements**
- C. They perform identically in all cases
- D. Neither supports insertion in the middle

40. What exception is thrown when you try to modify a list while iterating over it with a standard iterator (without using iterator.remove())?

- A. NoSuchElementException
- B. ConcurrentModificationException**
- C. UnsupportedOperationException
- D. IllegalStateException

41. Which interface provides key-value pair storage in Java Collections?

- A. List
- B. Set**
- C. Map**
- D. Queue

Set ==> If what is meant is (collection interface) in terms of the hierarchy

Map ==> (Collection Framework)

42. What does `Collections.unmodifiableList(list)` do?

- A. Creates a deep copy that is sorted
- B. Wraps the list so that structural modification attempts throw `UnsupportedOperationException`**
- C. Removes duplicates from the list
- D. Makes the list thread-safe

43. Which data structure would you choose for a FIFO (first-in-first-out) processing order?

- A. Stack
- B. Queue**
- C. TreeSet
- D. HashMap

44. What is the time complexity of retrieving a value by key from a well-implemented HashMap, on average?

- A. $O(n)$
- B. $O(\log n)$
- C. $O(1)$**
- D. $O(n \log n)$

45. Which method is used to iterate over all entries of a Map?

- A. `map.values()` only
- B. `map.entrySet()`**
- C. `map.keys()`
- D. `map.iterator()`

46. What happens when you put a key that already exists into a HashMap?

- A. A `DuplicateKeyException` is thrown
- B. The old value is replaced with the new value**
- C. Both values are stored under that key
- D. The put operation is silently ignored

47. Which Collection type does NOT allow duplicate elements?

- A. List
- B. Set**
- C. Queue
- D. ArrayList

48. What is the purpose of generics (e.g., `List`) in Java Collections?

- A. To improve runtime performance of loops
- B. To provide compile-time type safety and avoid explicit casting**
- C. To allow storing multiple types in the same list safely
- D. To make collections synchronized

Section 4: `equals()` and `hashCode()`

49. What is the general contract requirement between `equals()` and `hashCode()`?

- A. They are unrelated and can be implemented independently
- B. If two objects are equal according to `equals()`, they must have the same `hashCode()`**
- C. If two objects have the same `hashCode()`, they must be equal
- D. `hashCode()` must always return a unique value for every object

50. What is the default implementation of `equals()` inherited from `Object`?

- A. Compares field values
- B. Compares reference (memory address) equality, same as `==`**
- C. Always returns true
- D. Compares `hashCode()` values only

بمثابة أمر
أي أن هذا
ما يجب على
المبرمج مراعاته

51. Why is it important to override hashCode() when you override equals()?

- A. It is only a style convention with no functional impact
- B. To keep the class Serializable
- C. To ensure the class works correctly with hash-based collections like HashMap and HashSet**
- D. To make the class immutable

52. If two objects are NOT equal according to equals(), what does the contract say about their hashCode() values?

- A. They must be different
- B. They are not required to be different (they may or may not collide)**
- C. They must always be equal
- D. hashCode() cannot be called on unequal objects

53. What problem can occur if you override equals() but forget to override hashCode()?

- A. A compile-time error occurs
- B. Objects that are equal() may end up in different hash buckets, breaking HashSet/HashMap lookups**
- C. The JVM automatically generates a correct hashCode()
- D. equals() will stop working entirely

54. Which of these is a required property of equals() according to its contract? (choose the best answer)

- A. It must be reflexive, symmetric, transitive, and consistent**
- B. It must always return true for objects of the same class
- C. It must throw an exception when compared with null
- D. It must compare only primitive fields

55. What should `x.equals(null)` return for any non-null object x, according to the contract?

- A. true
- B. false**
- C. It should throw a NullPointerException
- D. It is undefined behavior

56. In a typical IDE-generated equals() method for a class with an int field 'age' and a String field 'name', what is usually checked first?

- A. Only field values, without any reference or type check
- B. Reference equality (this == obj) and null/class checks, then field comparisons**
- C. Only the hashCode() of both objects
- D. Only the class name as a String

57. Why is using mutable fields in hashCode()/equals() risky when the object is used as a HashMap key?

- A. It is not risky at all
- B. If the field changes after insertion, the object may become unfindable in the map
- C. It will cause a compile-time error
- D. HashMap automatically re-hashes on every access**

58. Which utility class provides helper methods like `Objects.equals(a, b)` and `Objects.hash(a, b, c)`?

- A. java.lang.Object
- B. java.util.Objects**
- C. java.lang.Class
- D. java.util.Arrays

59. What does `Objects.equals(a, b)` handle gracefully compared to calling `a.equals(b)` directly?

- A. Nothing extra, they behave identically in all cases
- B. It safely handles the case where 'a' is null, avoiding a `NullPointerException`**
- C. It ignores type mismatches
- D. It always returns true if either is null

60. If a class does not override `equals()` and `hashCode()`, what is used when it's placed as a key in a `HashMap`?

- A. A compile-time error occurs
- B. The inherited `Object` identity-based `equals()` and `hashCode()`, based on reference/memory identity**
- C. Java throws a runtime exception automatically
- D. All objects are treated as equal

Answer Key

Q#	Answer	Q#	Answer	Q#	Answer	Q#	Answer
1	B	16	B	31	B	46	B
2	B	17	C	32	C	47	B
3	B	18	C	33	C	48	B
4	B	19	B	34	B	49	B
5	C	20	C	35	C	50	B
6	B	21	A	36	B	51	C
7	B	22	C	37	C	52	B
8	C	23	A	38	B	53	B
9	B	24	B	39	B	54	A
10	B	25	B	40	B	55	B
11	B	26	A	41	C	56	B
12	C	27	B	42	B	57	B
13	A	28	B	43	B	58	B
14	B	29	C	44	C	59	B
15	B	30	B	45	B	60	B